React 19
Coding Interview Preparation Guide

Practical Coding Questions with Full Answers

With Setup Instructions, Full Code Answers & Hinglish Tips

API Integration | Hooks | Performance | Patterns | React Query | Zustand

Section 1 - React 19 New Features

Q01 Create a custom Input component in React 19 that accepts a ref without using forwardRef.

SETUP

Create a new React 19 project (Vite):

```
npm create vite@latest my-app -- --template react
cd my-app && npm install
npm run dev
```

TASK

1. Create Input.jsx that accepts ref as a direct prop.
2. In App.jsx, use a ref to focus the input on a button click.
3. No forwardRef -- that is removed in React 19.

ANSWER

```
// Input.jsx
export function Input({ ref, placeholder, ...props }) {
  return <input ref={ref} placeholder={placeholder} {...props} />;
}
// App.jsx
import { useRef } from 'react';
import { Input } from './Input';
export default function App() {
  const inputRef = useRef(null);
  return (
    <div>
      <Input ref={inputRef} placeholder='Type here...' />
      <button onClick={() => inputRef.current.focus()}>
        Focus Input
      </button>
    </div>
  );
}
```

INTERVIEW TIP (Hinglish)

> React 19 mein ref directly prop ki tarah aata hai. forwardRef() likhne ki zaroorat nahi -- bas function signature mein { ref, ...props } likho.

Q02 Build a component that uses the new use() hook to read a Context value conditionally.

SETUP

Same React 19 project.

TASK

1. Create ThemeContext using createContext.
2. Build a ThemedBox component that reads theme using use() -- not useContext.
3. Conditionally show theme color based on a prop 'show'.

ANSWER

```
// ThemeContext.js
import { createContext } from 'react';
export const ThemeContext = createContext('light');
// ThemedBox.jsx
import { use } from 'react';
import { ThemeContext } from './ThemeContext';
export function ThemedBox({ show }) {
  if (!show) return <p>Hidden</p>;
  const theme = use(ThemeContext); // conditional call -- allowed!
  return (
    <div style={{ background: theme === 'dark' ? '#1e293b' : '#f8fafc',
                  color: theme === 'dark' ? 'white' : 'black',
                  padding: 16 }}>
      Current theme: {theme}
    </div>
  );
}
// App.jsx
import { ThemeContext } from './ThemeContext';
import { ThemedBox } from './ThemedBox';
export default function App() {
  return (
    <ThemeContext value='dark'>
      <ThemedBox show={true} />
    </ThemeContext>
  );
}
```

INTERVIEW TIP (Hinglish)

> use() ka sabse bada advantage -- conditionally call ho sakta hai. Purana useContext sirf top-level mein call hota tha. Context provider ka syntax bhi React 19 mein hai, nahi.

Q03 Implement a Like button using useOptimistic that instantly updates count before server responds.

SETUP

React 19 project. Simulate server with a setTimeout.

TASK

1. Show a like count from initial prop.
2. On click -- instantly show +1 using useOptimistic.

3. Simulate a server call with 1.5s delay.

4. If server fails, count must rollback automatically.

ANSWER

```
import { useOptimistic, useState } from 'react';
async function fakeLikeAPI(shouldFail = false) {
  await new Promise(r => setTimeout(r, 1500));
  if (shouldFail) throw new Error('Server error');
}
export default function LikeButton({ initialLikes = 42 }) {
  const [likes, setLikes] = useState(initialLikes);
  const [optimisticLikes, addOptimistic] = useOptimistic(
    likes,
    (current, delta) => current + delta
  );
  async function handleLike() {
    addOptimistic(1);        // Instant UI update
    try {
      await fakeLikeAPI();   // Server call
      setLikes(l => l + 1); // Commit if success
    } catch {
      // React auto-rolls back optimistic update
      alert('Failed! Count rolled back.');
    }
  }
  return (
    <button onClick={handleLike}>
      Like {optimisticLikes} Likes
    </button>
  );
}
```

INTERVIEW TIP (Hinglish)

> useOptimistic automatically rollback karta hai jab async action complete hota hai -- manually kuch undo nahi karna padta. addOptimistic call karo, server fail ho toh original likes state wapas aa jaata hai.

Section 2 - API Integration

Q04 Fetch a list of users from jsonplaceholder and display them. Handle loading and error states properly.

SETUP

React 19 project (Vite). No extra packages needed.

TASK

1. Fetch users on component mount using useEffect.
2. Show a loading spinner while fetching.
3. Show an error message if fetch fails.
4. Cancel the fetch if component unmounts (AbortController).

ANSWER

```
import { useState, useEffect } from 'react';
export default function UserList() {
  const [users, setUsers]     = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError]     = useState(null);
  useEffect(() => {
    const controller = new AbortController();
    async function fetchUsers() {
      try {
        const res = await fetch(
          'https://jsonplaceholder.typicode.com/users',
          { signal: controller.signal }
        );
        if (!res.ok) throw new Error('Network error');
        const data = await res.json();
        setUsers(data);
      } catch (err) {
        if (err.name !== 'AbortError') setError(err.message);
      } finally {
        setLoading(false);
      }
    }
    fetchUsers();
    return () => controller.abort(); // cleanup
  }, []);
  if (loading) return <p>Loading...</p>;
  if (error)   return <p>Error: {error}</p>;
  return (
    <ul>
      {users.map(u => (
        <li key={u.id}>{u.name} -- {u.email}</li>
      ))}
    </ul>
  );
}
```

INTERVIEW TIP (Hinglish)

> useEffect ke andar async function directly mat likhna -- useEffect(async()=>{}) React warning deta hai. Andar define karke call karo. AbortController se memory leak avoid hota hai.

Q05 Build a user list with React Query -- with caching, loading state, and a refetch button.

INSTALL

```
npm install @tanstack/react-query
```

SETUP

React 19 + React Query setup:

```
npm install @tanstack/react-query
```

TASK

1. Wrap App with QueryClientProvider.
2. Use useQuery to fetch users from jsonplaceholder.
3. Show loading, error, and data states.
4. Add a Refetch button.
5. Set staleTime to 60 seconds.

ANSWER

```
// main.jsx
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
const queryClient = new QueryClient();
root.render(
  <QueryClientProvider client={queryClient}>
    <App />
  </QueryClientProvider>
);
// UserList.jsx
import { useQuery } from '@tanstack/react-query';
const fetchUsers = () =>
  fetch('https://jsonplaceholder.typicode.com/users')
  .then(r => { if (!r.ok) throw new Error('Failed'); return r.json(); });
export default function UserList() {
  const { data, isLoading, isError, error, refetch, isFetching } =
    useQuery({
      queryKey: ['users'],
      queryFn: fetchUsers,
      staleTime: 60 * 1000,
    });
  if (isLoading) return <p>Loading...</p>;
  if (isError)   return <p>Error: {error.message}</p>;
  return (
    <div>
      <button onClick={() => refetch()} disabled={isFetching}>
        {isFetching ? 'Refreshing...' : 'Refetch'}
      </button>
      <ul>
        {data.map(u => <li key={u.id}>{u.name}</li>)}
      </ul>
    </div>
  );
}
```

INTERVIEW TIP (Hinglish)

> queryKey array mein jo value ho woh caching key hai. staleTime matlab kitne time tak data fresh maana jaaye -- is time mein refetch nahi hoga even if component remounts.

Q06 Create a form to add a new post using useMutation (React Query). On success, refresh the posts list.

INSTALL

```
npm install @tanstack/react-query
```

SETUP

React Query already installed. Use jsonplaceholder POST endpoint.

TASK

1. useQuery se posts fetch karo.

2. useMutation se new post create karo (POST /posts).

3. On success -- invalidate 'posts' query to refetch.

4. Show pending state on submit button.

ANSWER

```
import { useQuery, useMutation, useQueryClient }
  from '@tanstack/react-query';
import { useState } from 'react';
const API = 'https://jsonplaceholder.typicode.com';
export default function Posts() {
  const queryClient = useQueryClient();
  const [title, setTitle] = useState('');
  const { data: posts = [] } = useQuery({
    queryKey: ['posts'],
    queryFn: () => fetch(`${API}/posts?_limit=5`).then(r => r.json()),
  });
  const mutation = useMutation({
    mutationFn: (newPost) =>
      fetch(`${API}/posts`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(newPost),
      }).then(r => r.json()),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['posts'] });
      setTitle('');
    },
  });
  function handleSubmit(e) {
    e.preventDefault();
    mutation.mutate({ title, body: 'New post', userId: 1 });
  }
  return (
    <div>
      <form onSubmit={handleSubmit}>
        <input value={title} onChange={e => setTitle(e.target.value)}
               placeholder='Post title' required />
        <button type='submit' disabled={mutation.isPending}>
          {mutation.isPending ? 'Saving...' : 'Add Post'}
        </button>
      </form>
      <ul>{posts.map(p => <li key={p.id}>{p.title}</li>)}</ul>
    </div>
  );
}
```

INTERVIEW TIP (Hinglish)

> invalidateQueries se React Query samajh jaata hai ki is queryKey ka data stale ho gaya -- next render mein automatically refetch karta hai. onSuccess mein invalidate karo, data turant fresh ho jaata hai.

Q07 Build a search input with API debouncing -- API call only after user stops typing for 500ms.

SETUP

React 19 project. No extra library needed -- implement debounce manually.

TASK

1. Input field for search query.
2. Debounce the API call by 500ms using useEffect + setTimeout.
3. Fetch from jsonplaceholder and filter by name.
4. Cancel previous timer on each keystroke.

ANSWER

```
import { useState, useEffect } from 'react';
export default function SearchUsers() {
  const [query, setQuery]   = useState('');
  const [results, setResults] = useState([]);
  const [loading, setLoading] = useState(false);
  useEffect(() => {
    if (!query.trim()) { setResults([]); return; }
    const timer = setTimeout(async () => {
      setLoading(true);
      try {
        const res = await fetch(
          'https://jsonplaceholder.typicode.com/users'
        );
        const data = await res.json();
        setResults(
          data.filter(u =>
            u.name.toLowerCase().includes(query.toLowerCase())
          )
        );
      } finally {
        setLoading(false);
      }
    }, 500);
    return () => clearTimeout(timer);
  }, [query]);
  return (
    <div>
      <input value={query} onChange={e => setQuery(e.target.value)}
             placeholder='Search users...' />
      {loading && <p>Searching...</p>}
      <ul>
        {results.map(u => <li key={u.id}>{u.name}</li>)}
      </ul>
    </div>
  );
}
```

INTERVIEW TIP (Hinglish)

> Debounce = useEffect cleanup ka smart use. Har keystroke par purana timer clear hota hai, naya set hota hai. 500ms koi key na dabaaye tab hi API call jaati hai. Yeh Google search jaise behaviour deta hai.

Q08 Create a reusable useFetch custom hook that handles loading, error, and abort.

SETUP

React 19 project. No extra packages.

TASK

1. Build useFetch(url) custom hook.
2. Returns { data, loading, error, refetch }.
3. Auto-cancel request on unmount using AbortController.
4. refetch() should re-trigger the API call.
5. Use it in a component to fetch posts.

ANSWER

```
// hooks/useFetch.js
import { useState, useEffect, useCallback } from 'react';
export function useFetch(url) {
  const [data, setData]       = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError]     = useState(null);
  const [tick, setTick]       = useState(0);
  useEffect(() => {
    const controller = new AbortController();
    setLoading(true);
    setError(null);
    fetch(url, { signal: controller.signal })
      .then(r => { if (!r.ok) throw new Error('Failed'); return r.json(); })
      .then(d => { setData(d); setLoading(false); })
      .catch(err => {
        if (err.name !== 'AbortError') {
          setError(err.message); setLoading(false);
        }
      });
    return () => controller.abort();
  }, [url, tick]);
  const refetch = useCallback(() => setTick(t => t + 1), []);
  return { data, loading, error, refetch };
}
// Posts.jsx
import { useFetch } from './hooks/useFetch';
export default function Posts() {
  const { data, loading, error, refetch } = useFetch(
    'https://jsonplaceholder.typicode.com/posts?_limit=5'
  );
  if (loading) return <p>Loading...</p>;
  if (error)   return <p>{error} <button onClick={refetch}>Retry</button></p>;
  return (
    <div>
      <button onClick={refetch}>Refresh</button>
      {data.map(p => <p key={p.id}>{p.title}</p>)}
    </div>
  );
}
```

INTERVIEW TIP (Hinglish)

> tick state refetch trigger karta hai -- jab bhi tick change hota hai useEffect re-run hota hai. Custom hooks mein 'use' prefix compulsory hai -- React lint rules check karta hai.

Section 3 - Hooks Deep Dive

Q09 Build a shopping cart using useReducer -- add item, remove item, clear cart, and show total.

SETUP

React 19 project. No extra packages.

TASK

1. Define cartReducer with actions: ADD_ITEM, REMOVE_ITEM, CLEAR.
2. Show list of products with Add to Cart buttons.
3. Show cart with quantities and total price.
4. Clear cart button.

ANSWER

```
import { useReducer } from 'react';
const products = [
  { id: 1, name: 'React Book', price: 499 },
  { id: 2, name: 'JS Course', price: 999 },
  { id: 3, name: 'Node Guide', price: 299 },
];
function cartReducer(state, action) {
  switch (action.type) {
    case 'ADD_ITEM': {
      const exists = state.find(i => i.id === action.item.id);
      if (exists)
        return state.map(i => i.id === action.item.id
          ? { ...i, qty: i.qty + 1 } : i);
      return [...state, { ...action.item, qty: 1 }];
    }
    case 'REMOVE_ITEM':
      return state.filter(i => i.id !== action.id);
    case 'CLEAR':
      return [];
    default: return state;
  }
}
export default function Cart() {
  const [cart, dispatch] = useReducer(cartReducer, []);
  const total = cart.reduce((s, i) => s + i.price * i.qty, 0);
  return (
    <div>
      <h3>Products</h3>
      {products.map(p => (
        <div key={p.id}>
          {p.name} - Rs.{p.price}
          <button onClick={() => dispatch({ type: 'ADD_ITEM', item: p })}>
            Add
          </button>
        </div>
      ))}
      <h3>Cart ({cart.length} items)</h3>
      {cart.map(i => (
        <div key={i.id}>
          {i.name} x{i.qty} = Rs.{i.price * i.qty}
          <button onClick={() => dispatch({ type: 'REMOVE_ITEM', id: i.id })}>
            Remove
          </button>
        </div>
      ))}
      <p>Total: Rs.{total}</p>
      <button onClick={() => dispatch({ type: 'CLEAR' })}>Clear Cart</button>
    </div>
  );
}
```

INTERVIEW TIP (Hinglish)

> useReducer mein dispatch reference stable hota hai -- useCallback mein wrap karne ki zaroorat nahi. Complex state (cart items with quantities) ke liye useReducer useState se zyada readable hota hai.

Q10 Implement a live search with useDeferredValue -- input stays responsive while results update.

SETUP

React 19 project. Create a large fake dataset.

TASK

1. Generate 10,000 fake items.
2. Input field for search -- must stay instantly responsive.
3. Use useDeferredValue to defer the filtering.
4. Show dim results while deferred value lags behind.

ANSWER

```
import { useState, useDeferredValue, useMemo } from 'react';
const items = Array.from({ length: 10000 }, (_, i) => ({
  id: i,
  name: `Item ${i} -- category ${i % 10}`,
}));
export default function SearchList() {
  const [query, setQuery] = useState('');
  const deferred = useDeferredValue(query);
  const isStale = deferred !== query;
  const filtered = useMemo(
    () => items.filter(i =>
      i.name.toLowerCase().includes(deferred.toLowerCase())
    ).slice(0, 50),
    [deferred]
  );
  return (
    <div>
      <input value={query} onChange={e => setQuery(e.target.value)}
             placeholder='Search 10,000 items...' />
      <p>{filtered.length} results {isStale ? '(updating...)' : ''}</p>
      <ul style={{ opacity: isStale ? 0.5 : 1 }}>
        {filtered.map(i => <li key={i.id}>{i.name}</li>)}
      </ul>
    </div>
  );
}
```

INTERVIEW TIP (Hinglish)

> useDeferredValue tab use karo jab tum parent ki value defer karna chahte ho. useTransition tab jab apni state update defer karni ho. Dono same React concurrent feature hain.

Q11 Build a form with useId for proper label-input accessibility linking.

SETUP

React 19 project.

TASK

1. Create a reusable FormField component.
2. Each FormField must link label to input using useId.
3. Render the same FormField twice -- each must have unique IDs.
4. Verify no htmlFor mismatch.

ANSWER

```
import { useId } from 'react';
function FormField({ label, type = 'text' }) {
  const id = useId();
  return (
    <div style={{ marginBottom: 12 }}>
      <label htmlFor={id} style={{ display: 'block', marginBottom: 4 }}>
        {label}
      </label>
      <input id={id} type={type} style={{ padding: 6, width: 200 }} />
    </div>
  );
}
export default function SignupForm() {
  return (
    <form>
      <FormField label='Full Name' type='text' />
      <FormField label='Email' type='email' />
      <FormField label='Password' type='password' />
      <button type='submit'>Sign Up</button>
    </form>
  );
}
```

INTERVIEW TIP (Hinglish)

> useId list keys ke liye mat use karo -- sirf accessibility IDs (htmlFor, aria-labelledby) ke liye hai. SSR mein bhi server aur client par same ID generate hoti hai -- hydration mismatch avoid hota hai.

Section 4 - Performance Optimization

Q12 Optimize a slow component using React.memo and useCallback -- prevent unnecessary re-renders.

SETUP

React 19 project.

TASK

1. Parent has a counter state and a list of items.
2. ExpensiveItem component logs 'rendered' on every render.
3. Wrap ExpensiveItem with React.memo.
4. Pass a delete handler using useCallback.
5. Verify ExpensiveItem does NOT re-render when counter changes.

ANSWER

```
import { useState, useCallback, memo } from 'react';
const ExpensiveItem = memo(function ExpensiveItem({ item, onDelete }) {
  console.log('ExpensiveItem rendered:', item.name);
  return (
    <li>
      {item.name}
      <button onClick={() => onDelete(item.id)}>Delete</button>
    </li>
  );
});
export default function App() {
  const [count, setCount] = useState(0);
  const [items, setItems] = useState([
    { id: 1, name: 'React' },
    { id: 2, name: 'Node.js' },
    { id: 3, name: 'TypeScript' },
  ]);
  const handleDelete = useCallback((id) => {
    setItems(prev => prev.filter(i => i.id !== id));
  }, []);
  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>
        Counter: {count}
      </button>
      <ul>
        {items.map(item => (
          <ExpensiveItem key={item.id} item={item} onDelete={handleDelete} />
        ))}
      </ul>
    </div>
  );
}
```

INTERVIEW TIP (Hinglish)

> React.memo + useCallback ek team hai. Sirf memo lagane se kaam nahi hoga agar handler function har render pe naya bane. useCallback stable reference deta hai tabhi memo skip kar pata hai re-render.

Q13 Implement code splitting with React.lazy and Suspense for a multi-page app.

SETUP

React 19 project (Vite -- already supports code splitting).

TASK

1. Create three pages: Home.jsx, About.jsx, Dashboard.jsx.
2. Lazy load all three with React.lazy.
3. Wrap routes with Suspense + loading fallback.
4. Verify in browser Network tab that each page JS loads on demand.

ANSWER

```
import { lazy, Suspense, useState } from 'react';
const Home      = lazy(() => import('./pages/Home'));
const About     = lazy(() => import('./pages/About'));
const Dashboard = lazy(() => import('./pages/Dashboard'));
function LoadingSpinner() {
  return <div style={{ padding: 20 }}>Loading page...</div>;
}
export default function App() {
  const [page, setPage] = useState('home');
  const pages = { home: Home, about: About, dashboard: Dashboard };
  const Page = pages[page];
  return (
    <div>
      <nav>
        {Object.keys(pages).map(p => (
          <button key={p} onClick={() => setPage(p)}
            style={{ fontWeight: page === p ? 'bold' : 'normal', marginRight: 8 }}>
            {p}
          </button>
        ))}
      </nav>
      <Suspense fallback={<LoadingSpinner />}>
        <Page />
      </Suspense>
    </div>
  );
}
```

INTERVIEW TIP (Hinglish)

> Vite automatically creates separate JS chunks for each lazy import. Network tab mein dekho -- Dashboard click karne par tabhi Dashboard chunk download hoga. Bundle size significantly kam hoti hai.

Q14 Build a virtual list that renders 50,000 items without freezing using react-window.

INSTALL

```
npm install react-window
```

SETUP

React 19 project + react-window:

```
npm install react-window
```

TASK

1. Generate 50,000 fake list items.
2. Render using FixedSizeList from react-window.
3. Each row shows item index and name.
4. Compare with normal map rendering (see the freeze).

ANSWER

```
import { FixedSizeList } from 'react-window';
const data = Array.from({ length: 50000 }, (_, i) => ({
  id: i,
  name: `Product ${i} -- Category ${i % 20}`,
}));
function Row({ index, style }) {
  return (
    <div style={{
      ...style,
      display: 'flex', alignItems: 'center',
      padding: '0 16px',
      borderBottom: '1px solid #e2e8f0',
      background: index % 2 === 0 ? '#f8fafc' : 'white',
    }}>
      <span style={{ color: '#94a3b8', marginRight: 12 }}>#{index + 1}</span>
      {data[index].name}
    </div>
  );
}
export default function VirtualList() {
  return (
    <div>
      <p>Showing 50,000 items (virtualized)</p>
      <FixedSizeList height={500} itemCount={50000} itemSize={48} width='100%'>
        {Row}
      </FixedSizeList>
    </div>
  );
}
```

INTERVIEW TIP (Hinglish)

> FixedSizeList sirf visible rows DOM mein rakhta hai -- 50,000 items mein se sirf ~10-12 rows. Scroll karne par rows recycle hote hain. Gmail, Notion, VS Code -- sab virtualization use karte hain.

Section 5 - Patterns and Advanced

Q15 Implement global auth state using Context API -- login, logout, and protected route.

SETUP

React 19 + React Router v6:

```
npm install react-router-dom
```

TASK

1. Create AuthContext with user, login(), logout().
2. Build a ProtectedRoute component that redirects to /login if not logged in.
3. Login page with fake credentials (admin / 1234).
4. Dashboard page visible only when logged in.

ANSWER

```
// AuthContext.jsx
import { createContext, useState, use } from 'react';
const AuthContext = createContext(null);
export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const login = (username, password) => {
    if (username === 'admin' && password === '1234') {
      setUser({ username, role: 'admin' });
      return true;
    }
    return false;
  };
  const logout = () => setUser(null);
  return (
    <AuthContext value={{ user, login, logout }}>
      {children}
    </AuthContext>
  );
}
export const useAuth = () => use(AuthContext);
// ProtectedRoute.jsx
import { Navigate } from 'react-router-dom';
import { useAuth } from './AuthContext';
export function ProtectedRoute({ children }) {
  const { user } = useAuth();
  return user ? children : <Navigate to='/login' replace />;
}
```

INTERVIEW TIP (Hinglish)

> Context value mein object dete waqt useMemo se wrap karo agar performance sensitive hai -- warna har parent render pe naya object banta hai aur saare consumers re-render karte hain.

Q16 Create a modal using ReactDOM.createPortal that renders outside the main app DOM tree.

SETUP

React 19 project. index.html mein ek extra div add karo.

TASK

1. index.html mein add karo.
2. Modal component createPortal use kare -- renders into #modal-root.
3. Open/close button in App.
4. ESC key press par modal close ho.
5. Backdrop click par bhi close ho.

ANSWER

```
// Modal.jsx
import { useEffect } from 'react';
import { createPortal } from 'react-dom';
export function Modal({ isOpen, onClose, children }) {
  useEffect(() => {
    function handleKey(e) {
      if (e.key === 'Escape') onClose();
    }
    if (isOpen) document.addEventListener('keydown', handleKey);
    return () => document.removeEventListener('keydown', handleKey);
  }, [isOpen, onClose]);
  if (!isOpen) return null;
  return createPortal(
    <div onClick={onClose} style={{
      position: 'fixed', inset: 0,
      background: 'rgba(0,0,0,0.5)',
      display: 'flex', alignItems: 'center', justifyContent: 'center',
    }}>
      <div onClick={e => e.stopPropagation()} style={{
        background: 'white', padding: 32, borderRadius: 12, minWidth: 300,
      }}>
        {children}
        <button onClick={onClose} style={{ marginTop: 16 }}>Close</button>
      </div>
    </div>,
    document.getElementById('modal-root')
  );
}
```

INTERVIEW TIP (Hinglish)

> Portal DOM mein #modal-root mein render hota hai lekin React tree mein App ka child hai. Isliye events React tree se bubble karte hain. e.stopPropagation() modal content mein lagao backdrop click se bachne ke liye.

Q17 Build an Error Boundary component that catches errors and shows a fallback UI.

SETUP

React 19 project.

TASK

1. Create ErrorBoundary class component.
2. BuggyComponent throws an error when a button is clicked.
3. ErrorBoundary should show friendly fallback UI.
4. Add a 'Try Again' button that resets the error state.

ANSWER

```
import { Component } from 'react';
class ErrorBoundary extends Component {
  state = { hasError: false, error: null };
  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }
  componentDidCatch(error, info) {
    console.error('Caught:', error, info.componentStack);
  }
  handleReset = () => {
    this.setState({ hasError: false, error: null });
  };
  render() {
    if (this.state.hasError) {
      return (
        <div style={{ padding: 24, border: '1px solid red', borderRadius: 8 }}>
          <h2>Something went wrong</h2>
          <p>{this.state.error?.message}</p>
          <button onClick={this.handleReset}>Try Again</button>
        </div>
      );
    }
    return this.props.children;
  }
}
function BuggyComponent() {
  const [crash, setCrash] = useState(false);
  if (crash) throw new Error('Intentional crash!');
  return <button onClick={() => setCrash(true)}>Click to crash</button>;
}
export default function App() {
  return (
    <ErrorBoundary>
      <BuggyComponent />
    </ErrorBoundary>
  );
}
```

INTERVIEW TIP (Hinglish)

> Error Boundary abhi bhi class component hona chahiye -- getDerivedStateFromError aur componentDidCatch lifecycle methods sirf class mein available hain. Reset ke liye state clear karo -- children fresh mount honge.

Section 6 - Forms and Zustand

Q18 Build a multi-step registration form using react-hook-form with validation.

INSTALL

```
npm install react-hook-form
```

SETUP

React 19 project + react-hook-form:

```
npm install react-hook-form
```

TASK

1. Step 1: Name + Email (required, email format validation).
2. Step 2: Password + Confirm Password (must match).
3. Next button disabled if current step is invalid.
4. On final submit -- log all form data.

ANSWER

```
import { useForm } from 'react-hook-form';
import { useState } from 'react';
export default function MultiStepForm() {
  const [step, setStep] = useState(1);
  const [allData, setAllData] = useState({});
  const { register, handleSubmit, watch,
    formState: { errors, isValid } } = useForm({ mode: 'onChange' });
  const onStep1 = (data) => {
    setAllData(d => ({ ...d, ...data }));
    setStep(2);
  };
  const onStep2 = (data) => {
    const final = { ...allData, ...data };
    console.log('All form data:', final);
    alert('Registration complete!');
  };
  if (step === 1) return (
    <form onSubmit={handleSubmit(onStep1)}>
      <h2>Step 1 of 2</h2>
      <input {...register('name', { required: 'Name required' })}
             placeholder='Full Name' />
      {errors.name && <p>{errors.name.message}</p>}
      <input {...register('email', {
        required: 'Email required',
        pattern: { value: /^[^\s@]+@[^\s@]+\.[^\s@]+$/,
                   message: 'Invalid email' }
      })} placeholder='Email' />
      {errors.email && <p>{errors.email.message}</p>}
      <button type='submit' disabled={!isValid}>Next</button>
    </form>
  );
  return (
    <form onSubmit={handleSubmit(onStep2)}>
      <h2>Step 2 of 2</h2>
      <input {...register('password', { required: true, minLength: 6 })}
             type='password' placeholder='Password' />
      <input {...register('confirm', {
        validate: v => v === watch('password') || 'Passwords do not match'
      })} type='password' placeholder='Confirm Password' />
      {errors.confirm && <p>{errors.confirm.message}</p>}
      <button type='submit' disabled={!isValid}>Register</button>
    </form>
  );
}
```

INTERVIEW TIP (Hinglish)

> react-hook-form uncontrolled inputs use karta hai -- sirf validation trigger hone par re-render hota hai. 100 fields bhi hoon toh performance issue nahi hoga. watch() se live field value milti hai -- confirm password match ke liye useful.

Q19 Build a global cart with Zustand -- add, remove, and persist to localStorage.

INSTALL

```
npm install zustand
```

SETUP

React 19 + Zustand:

```
npm install zustand
```

TASK

1. Create a Zustand store for cart -- items, addItem, removeItem, clearCart.
2. Persist cart to localStorage using zustand/middleware persist.
3. ProductCard component with Add to Cart.
4. CartSummary component showing count and total.
5. No Context Provider needed -- Zustand works without it.

ANSWER

```
// store/cartStore.js
import { create } from 'zustand';
import { persist } from 'zustand/middleware';
export const useCartStore = create(persist(
  (set, get) => ({
    items: [],
    addItem: (product) => set(state => {
      const exists = state.items.find(i => i.id === product.id);
      if (exists)
        return { items: state.items.map(i =>
          i.id === product.id ? { ...i, qty: i.qty + 1 } : i) };
      return { items: [...state.items, { ...product, qty: 1 }] };
    }),
    removeItem: (id) => set(state => ({
      items: state.items.filter(i => i.id !== id),
    })),
    clearCart: () => set({ items: [] }),
    total: () => get().items.reduce(
      (sum, i) => sum + i.price * i.qty, 0
    ),
  }),
  { name: 'cart-storage' }
));
// CartSummary.jsx
import { useCartStore } from './store/cartStore';
export function CartSummary() {
  const items     = useCartStore(s => s.items);
  const total     = useCartStore(s => s.total);
  const clearCart = useCartStore(s => s.clearCart);
  return (
    <div>
      <p>Items: {items.length} | Total: Rs.{total()}</p>
      <button onClick={clearCart}>Clear</button>
    </div>
  );
}
```

INTERVIEW TIP (Hinglish)

> Zustand mein subscribe selective hoti hai -- useCartStore(s => s.items) se sirf items change hone par re-render hoga. Context ke unlike koi Provider nahi chahiye -- store directly import karo kahin bhi.